

Module 1

Design & Verification Methodology (Part 1)

Learning objectives

- Spec-to-RTL flow: From SPEC.md (or equivalent) to synthesizable Verilog/SystemVerilog.
- Why verify: The cost of bugs, and how directed tests (and later UVM) reduce risk.
- Minimal toolchain: Verilator + Make + C++ harness for RTL simulation (no UVM in this module).

Prerequisites

- Linux/macOS/WSL2 with a terminal
- Verilator (5.x recommended)
- GNU Make
- C++ compiler (GCC or Clang)

Learning path

Learning Path

Diagram: Learning Path

(render with mmdc when available)

flowchart LR

T1["Specification → RTL Design F"]

T2["Design Methodology"]

T1 --> T2

T3["Intro to Verification: What "]

T2 --> T3

Understand the specification → RTL design flow, design methodology, and why we verify.

Overview

- Module 1 establishes the ****design and verification mindset**** you will use for UART, SPI, and I²C in...

How to learn this module

Suggested learning path

- Read UNDERSTANDING_THE_SPEC.md — how to read interface, behavior, and timing in a spec.
- Read SPEC_TO_RTL_GUIDE.md — mapping spec sections to RTL structure.
- Skim Design Architecture and Verification sections below, then run the spec_to_rtl example.
- Trace one requirement in SPEC.md to a line in ``dut/counter.v`` and to a check in ``sim_main.cpp``.

Design architecture

1. Artifact flow (spec_to_rtl)

- SPEC.md → requirements in plain language (interface, behavior, timing).
- dut/counter.v → synthesizable RTL implementing the spec.
- top.v → wrapper connecting DUT pins to the simulation harness.
- sim_main.cpp → clock, reset, stimulus, and result checking outside RTL.

```
Rtl Architecture
Diagram: Rtl Architecture
(render with mmdc when available)
flowchart TB
subgraph tb["Testbench (C++)"]
SIM["sim_main.cpp\nclk, rst_n, enable"]
end
subgraph rtl["RTL (top.v)"]
DUT["dut/counter.v\n8-bit up-counter"]
end
```

2. Counter DUT structure

- 8-bit up-counter: increments when `enable` is high; holds when disabled.
- Reset: synchronous active-low
`rst\_n` clears `count` to zero.
- Ports: `clk`, `rst\_n`, `enable`,
`count[7:0]` — map directly from the spec interface table.

Verification Architecture

```
Diagram: Verification Architecture
(render with mmdc when available)
flowchart LR
S["Directed stimulus\nreset, enable N cycles"] --> DUT["counter RTL"]
DUT --> C["Check count == N\n(PASS/FAIL in sim_main)"]
SPEC["SPEC.md"] -->|traceability| C
```

3. Simulation harness attachment

- C++ generates `clk` and drives `rst\_n` / `enable`; reads `count` each cycle.
- No SystemVerilog testbench yet — you will add structured TB/UVM in Module 2.

Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart LR

SPEC["SPEC.md"] --> RTL["counter RTL"]

RTL --> SIM["sim_main.cpp\nstimulus"]

SIM --> CHK["count == expected"]

CHK --> PASS["PASS / FAIL"]

RTL block diagram (reference)

Rtl Architecture

Diagram: Rtl Architecture

(render with mmdc when available)

flowchart TB

subgraph tb["Testbench (C++)"]

SIM["sim_main.cpp\nclk, rst_n, enable"]

end

subgraph rtl["RTL (top.v)"]

DUT["dut/counter.v\n8-bit up-counter"]

Module 1: DUT hierarchy and signal flow.

counter.v — spec in RTL

```
// 8-bit up-counter – RTL implementation per SPEC.md
// Module 1: Spec → RTL methodology example.

`timescale 1ns/1ps

module counter (
    input  wire      clk,
    input  wire      rst_n,
    input  wire      enable,
    output reg [7:0] count
);
    always @(posedge clk) begin
        if (!rst_n)
            count <= 8'h00;
        else if (enable)
            count <= count + 8'd1;
    end
endmodule
```

...

Execution & simulation flow

How the example runs (toolchain)

- Makefile: Verilator compiles RTL + SystemVerilog testbench into a C++ model
- sim_main.cpp: generates clk/rst_n, calls eval() until \$finish
- Directed test (initial block or C++): drive stimulus, wait for DUT flags
- Self-check: compare outputs; print PASS/FAIL (see terminal demo slide)
- Repo path: module1/examples/<example>/ — run make run from that directory

Makefile — Verilator build & run

```
# Makefile for Module 1 spec_to_rtl example
# Spec → RTL → basic simulation (no UVM).
#
# Usage:
#   make run           Build and run (default)
#   make run TRACE=1   Build with VCD tracing, then run
#   make clean         Remove obj_dir and VCD
```

```
TRACE ?= 0
VERILATOR ?= verilator
TOPLEVEL ?= top
DUT_SRC = dut/*.v
RTL_SRC = top.v $(DUT_SRC)
HARNESS = sim_main.cpp
```

```
VERILATOR_FLAGS = -cc --exe -Wall
```

```
# ...
```

Key files to study

Open these in the repo

- ``module1/examples/spec_to_rtl/SPEC.md`` — requirements document
- ``module1/examples/spec_to_rtl/dut/counter.v`` — RTL implementation
- ``module1/examples/spec_to_rtl/top.v`` — DUT wrapper
- ``module1/examples/spec_to_rtl/sim_main.cpp`` — stimulus and checking

Verification & testing methods

1. What we verify in Module 1

- RTL behavior matches SPEC.md
(reset, enable gating, count progression).
- Directed scenario: release reset, enable for N cycles, expect `count == N`.

Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart LR

SPEC["SPEC.md"] --> RTL["counter RTL"]

RTL --> SIM["sim_main.cpp\nstimulus"]

SIM --> CHK["count == expected"]

CHK --> PASS["PASS / FAIL"]

2. Stimulus and checking

- Stimulus: C++ toggles `enable` for a known number of clock cycles.
- Check: compare `count` to expected value; print PASS/FAIL.
- Traceability: each check should cite the spec sentence it proves.

Testing Methods

```
Diagram: Testing Methods
(render with mmdc when available)
flowchart LR
SPEC["SPEC.md"] --> RTL["counter RTL"]
RTL --> SIM["sim_main.cpp\nstimulus"]
SIM --> CHK["count == expected"]
CHK --> PASS["PASS / FAIL"]
```

3. What is not covered yet

- Random stimulus, functional coverage, scoreboards — Module 2 (UVM) and protocol modules.
- Use [UNDERSTANDING_THE_SPEC.md] (

../module1/UNDERSTANDING_THE_SPE
C.md) and
[SPEC_TO_RTL_GUIDE.md](.....

Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart LR

SPEC["SPEC.md"] --> RTL["counter RTL"]

RTL --> SIM["sim_main.cpp\nstimulus"]

SIM --> CHK["count == expected"]

CHK --> PASS["PASS / FAIL"]

sim_main.cpp — directed check

```
// C++ test harness for spec_to_rtl (8-bit counter).  
// Drives clk, rst_n, enable and checks count behavior.  
// Module 1: basic directed test — no UVM.
```

```
#include <cstdint>  
#include <cstdlib>  
#include <iostream>
```

```
#include <verilated.h>  
#include "Vtop.h"
```

```
#if VM_TRACE  
# include <verilated_vcd_c.h>  
#endif
```

```
static constexpr int kClockHalfPeriod = 5;  
# ...
```

Syllabus topics

1. Specification → RTL Design Flow

- Specification: Document interface (ports, widths, direction), behavior (reset, enable, state transi...
- RTL: Implement the spec in Verilog/SystemVerilog; each requirement in the spec should map to clear...
- Traceability: Being able to point from a line in the spec to the RTL that implements it (and vice v...

2. Design Methodology

- Single block: One spec, one (or a few) RTL files, one testbench.
- Reuse: The same flow (spec → RTL → testbench) scales to protocol blocks (UART, SPI, I²C) in Modules...
- Documentation: Keep SPEC.md (or similar) next to the RTL so that reviewers and verification can che...

3. Intro to Verification: What We Verify, and Why

- What we verify: That the RTL matches the specification (correct function, reset, enable, boundaries...
- Why we verify: Finding bugs in RTL is far cheaper than finding them in silicon or in system integra...
- Directed test: A simple test that drives specific inputs (e.g., reset, then 10 enabled cycles) and...

Command reference highlights

Environment checks

- `verilator --version`

Build and run spec_to_rtl

- `cd module1/examples/spec_to_rtl`

Module script (from repo root)

- `./scripts/module1.sh --check # Environment only`

Hands-on examples

Module 1 self-check

```
$ ./scripts/module1.sh --help
```

Terminal: module1_check

```
[[0;36m===== [[0m
[[0;36mModule 1: Demo commands (copy-paste to try) [[0m
[[0;36m===== [[0m

From repo root:

# 1) Environment sanity:
verilator --version
make --version

# 2) Run the spec_to_rtl example (spec → RTL → simulation):
cd module1/examples/spec_to_rtl
make run

# 3) Optional: with VCD tracing:
make run TRACE=1
# Then open spec_to_rtl.vcd in a waveform viewer (e.g. gtkwave).

[[1;33m[INFO] [[0m See module1/EXAMPLES.md for more.
```

Example 1: Spec to RTL counter

- Reading SPEC.md and mapping each requirement to RTL in ``dut/counter.v``
- Building a minimal top and C++ harness (clock, reset, enable)
- Directed verification: enable for N cycles and check ``count == N``

Demo: Spec to RTL counter

```
$ cd module1/examples/spec_to_rtl && make run
```

Terminal: ex01_spec_to_rtl

```
```bash  
make run
```
```

Expected: `[PASS] spec\_to\_rtl: counter behavior matches spec.`

With VCD tracing (optional):

Key concepts

1. Specification first

- Interface, behavior, and timing live in SPEC.md before RTL.
- Every check in the testbench should trace to a spec sentence.

2. Traceability

- Spec line ↔ RTL line ↔ test expectation — practice for UART/SPI/I²C later.

Common pitfalls

RTL before spec

- Mistake: Coding counters or FSMs without a written spec.
- Correct: Complete SPEC.md (or equivalent) first.
- Why: Reviewers and tests have no golden reference.

Weak checks

- Mistake: Only printing values without comparing to expected results.
- Correct: Explicit PASS/FAIL when ``count != expected``.
- Why: Simulation can run without verifying correctness.

Practice & assessment

What you should know

- Describe the specification → RTL flow and why a written spec matters.
- Explain what we verify (RTL vs spec) and why (cost of bugs, repeatability).
- Run the spec_to_rtl example (make run) and relate SPEC.md, counter.v, and sim_main.cpp.
- Be ready for Module 2: basic testbench patterns and UVM+SV (agents, sequences, drivers, monitors, s...

Exercises

- Read the spec and RTL
- Run and extend the test
- Optional: Trace

Assessment checklist

- Can explain the spec → RTL flow and the role of a written specification.
- Can explain what we verify (RTL vs spec) and why verification is done.
- Can run the spec_to_rtl example and describe what SPEC.md, counter.v, and sim_main.cpp do.
- Ready to move to Module 2 (basic TB, UVM+SV, toolchain).

Summary & next steps

- Understand the specification → RTL design flow, design methodology, and why we verify.
- Complete module1/CHECKLIST.md
- Review module1/EXAMPLES.md and run each lab
- Next: Design & Verification Methodology (Part 2)